

Web Security Fundamentals

Four real applications, four broken trust boundaries

Bahati · Saturday 3 October 2026

Everything in this talk is public and in the repository, including the source of every application, the reference solutions, and the verification script:

<https://github.com/meshackbahati/web-sec-fund>

Authorised testing only. The four applications in this talk are deliberately vulnerable, deployed by me for this demonstration, and contain no real data.

The plan

1. **Definitions**, the vocabulary up front
2. **The one idea** behind all four bugs
3. **Four applications**, one each:
 - Session Forge, a forged identity
 - Clearance, a query you should not have been able to write
 - Reflector, code running in someone else's browser
 - Keyring, a lock that accepts the wrong key
4. **The fixes**, one line each
5. **Defending properly**

Everything is demonstrated live. Nothing is simulated.

Definition: web application

A **web application** is a program you talk to over the network using HTTP.

Two sides:

- The **client**, usually a browser. It sends requests and renders what comes back.
- The **server**, your code. It receives the request, decides what to do, and returns a response.

```
graph LR; browser -- "GET /catalogue?category=Home" --> server; server -- "200 OK, HTML" --> browser;
```

The diagram illustrates the communication between a browser and a server. On the left, the word "browser" is followed by a horizontal line. This line connects to the text "GET /catalogue?category=Home". Another horizontal line connects this text to a right-pointing arrow, which then points to the word "server". Below this, a left-pointing arrow connects the "server" back to a horizontal line, which then connects to the text "200 OK, HTML", which is finally connected to the "browser" by another horizontal line.

Definition: HTTP request

A **request** is a message from client to server with four parts that matter:

Part	Example	Meaning
Method	GET	What kind of action: fetch, submit, delete
Path	/admin	Which resource
Headers	Cookie: session=...	Metadata, including who you claim to be
Body	username=wiener	Data being sent

A **response** carries a **status code**: 200 success, 302 redirect, 403 forbidden, 500 server error.

Definition: untrusted input

Untrusted input is any value that reached your server without your code creating it.

Form fields, query strings, cookies, headers, JSON bodies, file names. All of it. So is anything derived from it.

| The rule this talk is built on: **input is data, never instructions.**

When a server lets input change the *meaning* of a query, a template, or a verification routine, the input has become instructions.

Definition: authentication and authorisation

Authentication: *who are you?*

Proves an identity. Password, multi-factor code, passkey.

Authorisation: *what may you do?*

Decides whether that identity may perform this action.

They are different questions, and confusing them is common:

- You can be **authenticated** and still **not authorised**. Signing in as `wiener` proves who you are; it does not give you the admin panel.
- Every one of my four demos is an **authorisation** failure, not an authentication failure. Nobody broke a password. They talked their way past a check.

Definition: session and cookie

The server must remember who you are across requests. HTTP does not do this natively. Each request stands alone.

Server-side session: the server keeps the state, the browser holds an opaque reference (a random string) in a **cookie**.

```
cookie: session=8f2a91c4...      ← meaningless on its own, a lookup key  
server: session store maps 8f2a91c4 → { user: wiener }
```

Stateless session (JWT): all the state travels *inside* the cookie.

Definition: JSON Web Token

A **JWT** is three base64url-encoded segments joined by dots.

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJ3aWVuZXIifQ . 7Vm1k2l...  
└── header ─┘      └── payload ─┘      └── signature ─┘
```

- **header**: metadata, including which algorithm signed it
- **payload**: the claims: `sub` (subject, the user), `exp` (expiry), `role`
- **signature**: proof the token was not altered

A **claim** is a key/value statement inside the payload. `sub: "wiener"` is a claim. So is `role: "administrator"`.

base64url is an *encoding*, not encryption. **Anyone holding a JWT can read it.** Open yours in any text editor.

Definition: base64url

An **encoding**: a reversible translation of bytes into text that survives being put in a URL or a cookie.

Ordinary base64 uses `+` and `/`, which are awkward in URLs, so base64url swaps them for `-` and `_` and drops the `=` padding.

It provides **no confidentiality whatsoever**. Its only job is transport.

Definition: HMAC and digital signature

A **digital signature** proves two things: the message came from the holder of a secret key, and it has not changed since.

HS256 is HMAC using SHA-256:

```
signing_input = "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ3aWVuZlZlZXIifQ"  
signature     = base64url( HMAC-SHA256( secret_key , signing_input ) )
```

To verify, the server recomputes the same value with its own key and compares it **in constant time**: a comparison that takes the same time whether it matches or not, so it cannot be timed to leak the answer.

Definition: algorithm confusion

Tokens may be signed two ways:

	Signs with	Verifies with	Both keys known to you?
HS256	one shared secret	same secret	only the server
RS256	private key	public key	the public key is public

Algorithm confusion happens when a server holding both key types decides which to use by reading the `alg` value *from the token itself*.

The attacker then presents an `HS256` token and supplies the **public** key as the HMAC secret. The server verifies with a key the attacker chose, and concludes the token is genuine.

The server must choose the algorithm. Never let the token decide.

The four targets

	Application	The one defect	Where it lives
01	Session Forge	trusts the <code>alg</code> inside the token	the token boundary
02	Clearance	concatenates input into SQL	the query boundary
03	Reflector	writes input into HTML unencoded	the rendering boundary
04	Keyring	verifies with a key the token carries	the key boundary

All four: Next.js, SQLite, deployed on Vercel, and reachable now.

Demo 1: Session Forge

<https://northwind-01-session-forge.vercel.app>

Sign in as a normal employee. The goal is the administration page.

Here is the real verification function, from

`apps/01-session-forge/lib/store.js` :

```
export function verifyToken(token) {
  /* decode header and payload */

  // VULNERABLE (application 01)
  if (header.alg === 'none') {
    const unsignedExpiry = Math.floor(Date.now() / 1000);
    if (typeof payload.exp === 'number' && payload.exp < unsignedExpiry) return null;
    return { ...payload, _verified: false };
  }

  if (header.alg !== 'HS256') return null; // everything else is pinned
}
```

What correct looks like

Every application is a copy of one reference implementation with exactly one marked substitution. The correct `verifyToken`, in `apps/_shared/store.js`:

```
// CORRECT: the server decides which algorithm is acceptable.
if (header.alg !== ACCEPTED_ALGORITHM) return null;

// CORRECT: raw bytes on both sides. Comparing the base64url text would
// compare a 43-byte string against a 32-byte MAC and never match.
const expected = createHmac(HMAC_ALGORITHM, signingSecret())
  .update(`${encodedHeader}.${encodedPayload}`)
  .digest();
const provided = Buffer.from(encodedSignature, 'base64url');
if (expected.length !== provided.length || !timingSafeEqual(expected, provided)) {
  return null;
}
```

Three things to notice: the algorithm is pinned by the server, the MAC is compared in constant time, and the comparison is on raw bytes.

Demo 1: the attack

Everything below is typed live. Watch the status codes.

```
BASE=https://northwind-01-session-forge.vercel.app
```

1. Sign in as a normal employee. The token arrives in the response:

```
curl -s -c jar.txt -o /dev/null -X POST \  
  -d 'username=wiener&password=peter' "$BASE/login"  
  
set-cookie: session=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJub3J0a...  
           Path=/; HttpOnly; SameSite=Lax; Secure
```

2. Replay that token against the admin page. The boundary works:

```
curl -s -b jar.txt -o /dev/null -w 'HTTP %{http_code}\n' "$BASE/admin"  
# HTTP 403
```

Demo 1: the fix

Delete the branch that trusts the token's `alg`:

```
if (header.alg !== 'HS256') return null;
```

Better still: pin the algorithm in the library's own options rather than in your code, and reject `none` explicitly.

Lesson: the server decides which algorithm verifies a token, never the token.

Definition: SQL and a query

SQL is the language databases are asked questions in.

```
SELECT sku, name, price FROM products WHERE category = 'Home';
```

- **SELECT ... FROM:** which columns, from which table
- **WHERE:** which rows
- **'Home'** : a **string literal**: text, quoted, not a command

A **table** is rows and columns; here, one row per product.

Definition: parameterisation

The safe way to filter. You send the **shape** of the query and the **value** separately:

```
db.prepare(  
    "SELECT ... FROM products WHERE category = ? AND classification = 'public'"  
)<all>(category);
```

The database receives:

- the statement, with `?` placeholders
- the value, `Home`

It parses the statement once and treats the value strictly as data. It is **never parsed as SQL**, no matter what it contains.

The unsafe way builds the sentence with the value pasted in:

Demo 2: Clearance

<https://northwind-02-clearance.vercel.app>

No login needed. The catalogue filter is the whole target. Built in

`apps/02-clearance/app/catalogue/route.js` :

```
// VULNERABLE (application 02)
const sql = `SELECT sku, name, category, price_cents, classification`
          + ` FROM products WHERE category = '${category}'`
          + ` AND classification = 'public' ORDER BY id`;
```

```
rows = db().prepare(sql).all();
```

```
``` Built in
```

```
`apps/02-clearance/app/catalogue/route.js`:
```

```
```js
```

```
// VULNERABLE (application 02)
```

```
const sql = `SELECT sku, name, category, price_cents, classification`
          + ` FROM products WHERE category = '${category}'`
```

Two details that cost us real time

`--` ends at the newline. When the two conditions sat on separate lines, the clearance filter survived the comment and the attack silently returned only public rows. It looked like the exploit had failed.

A `UNION` needs matching column counts or SQLite rejects the whole statement, so the injected `SELECT` must project exactly as many columns as the original.

That second one turns a bypass into arbitrary read:

```
x' UNION SELECT sql,name,type,name,0 FROM sqlite_master WHERE type='table'--
```

That reads the database's own schema. Arbitrary read, not just a bypass.

Demo 2: the fix

```
const sql = `SELECT ... WHERE category = ? AND classification = 'public' ORDER BY id`;
rows = db.prepare(sql).all(category);
```

One line. The input becomes data again.

An allow-list of permitted categories is a useful **second** layer.

Parameterisation is the actual control.

"We sanitised it" is not a fix. Escaping quotes does not help, and blacklisting `OR` and `UNION` is a losing game.

Also: this app returns the database error to the user. That is a second finding. It turns a blind injection into a verbose one.

Definition: HTML and parsing

HTML is a text format that a browser **parses** into a document.

```
<p class="name">Enamel Kettle</p>
```

▲ ▲
element attribute

The parser has one job: decide what is **markup** and what is **text**.

It has no way to know which characters you *meant* as markup. If a template writes user input straight into the page, the parser simply finds a `<script>` element and runs it.

Definition: output encoding

Encoding replaces characters that mean something structural with inert equivalents:

You write	User sees	Parser sees
<code><script></code>	<code><script></code>	text, not an element
<code>&</code>	<code>&</code>	not the start of an entity

```
escapeHtml('<script>')    // '&lt;script&gt;'
```

This restores the distinction the parser lost. It is **context-specific**: the correct encoding for HTML text is not the one for a JavaScript string, an attribute value, or a URL.

Demo 3: Reflector

<https://northwind-03-reflector.vercel.app>

`app/search/route.js` contains both renderings of the same value, some lines apart:

```
// correct: the search box
<input type="search" name="q" value="{escapeHtml(query)}" ...>

// VULNERABLE (application 03): the results heading
<h2>{rows.length} results for &ldquo;{query}&rdquo;</h2>
```

One is escaped. The other is not. The browser cannot tell which characters were meant as text.

First, from a terminal. This is what an attacker gets:

```
BASE=https://northwind-03-reflector.vercel.app
```

The shape of the attack

Notice what the attacker did **not** do:

- did not steal a password
- did not obtain the administrator's session cookie
- did not send a single request to the console

The link carries a payload. An administrator opens it. The script runs **in their browser**, with their session, and calls the console as them.

That is what cross-site scripting means: running code in someone else's browser, with their privileges.

A subtlety worth naming

The session cookie in these applications is marked `HttpOnly`, which stops `document.cookie` from reading the token in JavaScript.

It does **not** stop this attack.

`HttpOnly` prevents cookie theft. It does not prevent the injected script from *acting as the user*. Both are real mitigations for different problems, and conflating them is common.

Demo 3: the fix

```
&ldquo;${escapeHtml(query)}&rdquo;
```

One call. Behind it, a second layer: a **Content Security Policy** forbidding inline script would have blocked this payload regardless of encoding.

Encoding is the fix. CSP is the belt.

Definition: JWK

A **JWK** (JSON Web Key) is a standard way to write a key as JSON.

```
{ "kty": "oct", "kid": "attacker", "k": "" }
```

The `kid` (**key ID**) names which key signed a token. A legitimate feature for servers holding several keys.

The mistake is trusting the token to say **which key verifies it**. An empty key is a perfectly valid HMAC secret, so a token signed with the empty string verifies.

Demo 4: Keyring

<https://northwind-04-keyring.vercel.app>

From `apps/04-keyring/lib/store.js` :

```
// VULNERABLE (application 04)
if (header.jwk && typeof header.jwk.k === 'string') {
  const embedded = createHmac(HMAC_ALGORITHM, header.jwk.k)
    .update(`${encodedHeader}.${encodedPayload}`)
    .digest();
  const given = Buffer.from(encodedSignature, 'base64url');
  if (embedded.length !== given.length || !timingSafeEqual(embedded, given)) return null;
  return { ...payload, _verified: false };
}
```

`BASE=https://northwind-04-keyring.vercel.app`

```
curl -s -c jar.txt -o /dev/null -X POST \
  -d 'username=wiener&password=peter' "$BASE/login"
```

```
curl -s -b jar.txt -o /dev/null -X 'HTTP %[http_code]\n' "$BASE/admin"
```


The same four, in Burp Suite

Everything just shown by hand is what a proxy makes routine.

	Where in Burp	What you do
01	JWT Editor extension (BApp store)	Re-encode the token in the message editor: <code>alg</code> to <code>none</code> , <code>sub</code> to <code>administrator</code> , send to Repeater
02	Repeater	Intercept <code>GET /catalogue</code> , edit the parameter, resend. <code>sqlmap -u ... --batch</code> also finds it unaided
03	Proxy history	Proxy the browser, open the crafted URL, and read the injected <code>fetch('/console')</code> in the history
04	JWT Editor extension	Add an arbitrary header claim carrying the <code>jwk</code> , resign with an empty key

Reading the results honestly

Attack	Unpatched	After the one-line fix
01 forged <code>alg:none</code> token	flag returned	rejected
02 <code>OR 1=1</code> tautology	flag returned	treated as data
03 reflected script	flag returned	encoded
04 embedded JWK	flag returned	key no longer chosen

Thirteen automated checks pass against the live deployments (`apps/solve/verify.sh`), including an assertion that the deployed builds are the **vulnerable** ones, so the demonstrations cannot silently stop working.

How to defend, in order of value

1. **Parameterise every query.** No exceptions, no "just this once".
2. **Encode output for its context.** Every time, at the point of output.
3. **Deny by default.** Check ownership server-side on every request.
4. **Pin token algorithms** in library configuration.
5. **Never let a token choose its own key.**
6. **Content Security Policy** as a second layer, not the first.
7. **Do not return errors to users.**
8. **Hash passwords** with argon2id or bcrypt. Not SHA-256. Not "base64".
9. **Keep secrets out of the repository** and rotate them.
10. **Log and monitor** authentication and authorisation decisions.

What to take away

- **Never trust a claim you have not verified**, including one the client wrote for you.
- The client is not part of your program. Everything it sends is input.
- **Authorisation is not authentication**. Signing in proves nothing about permission.
- Encoding and parameterisation are not optional extras. They are the control.
- Defence in depth: encoding *and* CSP, authorisation *and* ownership checks.
- Most of these bugs are one line long. That is why review matters.

If you take one thing

| Input is data. Never instructions.

Everything in the next hour is a consequence of breaking that rule: in a token header, in a SQL string, in an HTML page, and in a key lookup.

Everything here is public

<https://github.com/meshackbahati/web-sec-fund>

In the repository	What it is
<code>apps/01-session-forge</code> ... <code>apps/04-keyring</code>	the four applications, with a <code>WRITEUP.md</code> beside each
<code>apps/CHALLENGES.md</code>	all four write-ups together, with the shared mechanisms
<code>apps/solve/</code>	the reference solutions and <code>verify.sh</code> , thirteen checks
<code>apps/build.mjs</code>	regenerates all four from one implementation
	deploys all four and sets their flags as

Resources

- **PortSwigger Web Security Academy**, free legal labs:
<https://portswigger.net/web-security>
- **OWASP Cheat Sheet Series**, the practical reference:
<https://cheatsheetseries.owasp.org>
- **RFC 7519 (JWT)** and **RFC 7515 (JWS)**:
<https://www.rfc-editor.org/rfc/rfc7519>
- **MDN**: HTTP, cookies, CSP, CORS:
<https://developer.mozilla.org>
- Live targets, one per application: `apps/deployments.md` in the repository

Thank you

Questions?

Authorised testing only. These applications exist to be attacked, contain real defects and no real data, and are removed once the event is over.
Never use these techniques against systems you have no permission to test.